ThreatLens

Database Setup Guide

A Cyber Threat Intelligence Indicator Quality Checker | ZEIT3118 Capstone | UNSW Canberra (ADFA)

ThreatLens extracts indicators of compromise (IoCs) from threat reports and feeds, organises them into a structured repository, and checks them for quality issues such as duplication, missing context, and likely false positives. This guide covers how to build the database on your own machine.

What you need first

Python 3.12 or newer, Git, and VS Code (recommended editor). Optionally DB Browser for SQLite (64-bit Windows build from sqlitebrowser.org/dl) for viewing the database.

> **No database server to install**
>
> The project uses SQLite, which is built into Python. The whole database lives in a single file — nothing to provision, no admin rights required.

Setup (first time)

Run these steps once, from the project root, to get a working database.

1. Clone the repository

```
git clone https://github.com/m0ngrl/ITP2-2026.git
cd ITP2-2026
```

2. Create and activate a virtual environment (Windows PowerShell)

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

If PowerShell blocks the activation script, either use Command Prompt (`.venv\Scripts\activate.bat`) or allow scripts for your user once with:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

You will know it worked when your prompt starts with (.venv).

3. Install the dependencies

```
pip install -e ".[dev]"
```

This installs the project itself in editable mode, along with SQLAlchemy and Alembic — both are declared as dependencies in `pyproject.toml`, so nothing else needs installing separately.

4. Build the database

The migration scripts live in `migrations/`, and the ORM models they build from live in `src/threatlens/db/models.py`. Run Alembic from the project root, pointing it at the migrations config:

```
alembic -c migrations/alembic.ini upgrade head
```

This creates `threatlens.db` inside the `db` folder with all seven tables.

5. Seed the reference data

Still from the project root, with the virtual environment active:

```
python -m threatlens.db.seed
```

This fills two tables the quality engine depends on: `scoring_config` (quality-dimension weights and scoring bands) and `allowlist` (known-benign entries like public DNS servers and private IP ranges). You should see it report 5 and 16 rows.

6. Confirm it worked

You now have a `db/threatlens.db` file. Open it in DB Browser for SQLite (Open Database, then select the file) and check the Database Structure tab shows all seven tables listed below. You will also see `alembic_version`, which is Alembic's own tracking table.

The seven tables at a glance

Table	Holds
sources	One row per report or feed ingested
raw_documents	Archived copy of each original report/feed
indicators	The deduplicated IoCs (IPs, domains, URLs, hashes)
sightings	Links each indicator to the sources it appeared in
quality_flags	Quality issues detected against each indicator
allowlist	Known-benign values for false-positive checks
scoring_config	Quality-dimension weights and scoring bands

The clean handover point is the indicators / sightings tables: the extraction engine fills them, the quality engine reads them.

Everyday commands

Always activate the virtual environment first (`.\.venv\Scripts\Activate.ps1`), and run every command below from the project root — not from inside `migrations/` or `db/`.

Task	Command (run from project root)
Apply latest schema	alembic -c migrations/alembic.ini upgrade head
Re-seed reference data	python -m threatlens.db.seed
Check current migration	alembic -c migrations/alembic.ini current

Task	Command (run from project root)
Create migration after editing models.py	alembic -c migrations/alembic.ini revision --autogenerate -m "describe change"

Important notes

> **Do not commit the database**
>
> threatlens.db (and its .db-shm / .db-wal companions) are excluded by .gitignore. Everyone builds their own from the migrations. What is version-controlled is the recipe, not the file.

> **Close DB Browser before running Python scripts**
>
> SQLite allows only one writer at a time, so an open DB Browser can cause a “database is locked” error. Click Write Changes or Revert Changes first.

> **Schema changes go through Alembic**
>
> Never edit the database directly in DB Browser. After changing models.py, generate a migration (alembic -c migrations/alembic.ini revision --autogenerate) and apply it (alembic -c migrations/alembic.ini upgrade head), then commit the new file in migrations/alembic/versions/.

> **Pull before you push**
>
> Run git pull before git push so you pick up each other’s changes. Never use git push --force on the shared repo.

Project structure

```
ITP2-2026/
|-- .gitignore
|-- pyproject.toml Declares sqlalchemy + alembic as dependencies
|-- README.md
|-- db/
| `-- threatlens.db Generated database file (gitignored, not committed)
|-- migrations/
| |-- alembic.ini Alembic configuration
| `-- alembic/
| |-- env.py Loads models from threatlens.db.models
| `-- versions/ Migration scripts that build the schema
`-- src/threatlens/db/
|-- models.py SQLAlchemy models - the seven-table schema
|-- seed.py Populates scoring_config and allowlist
`-- __init__.py
```

Troubleshooting

Symptom	Fix
ModuleNotFoundError: No module named 'threatlens'	The project isn't installed. Run pip install -e ".[dev]" from the project root with the venv active.
Activate.ps1 will not run	PowerShell execution policy — see step 2.
“database is locked”	DB Browser has the file open with pending changes. Click Write Changes or Revert Changes, or close DB Browser, then retry.
upgrade head does nothing / tables missing	Check you're running commands from the project root and that alembic -c migrations/alembic.ini current shows a revision. While there is no real data, you can delete db/threatlens.db and run alembic -c migrations/alembic.ini upgrade head then python -m threatlens.db.seed to rebuild from scratch.